

Network Layer Security_ IKE

Module 3

Prepared by

Jesna J S

Assistant Professor, CSE TKMCE

INTERNET KEY EXCHANGE (IKE)

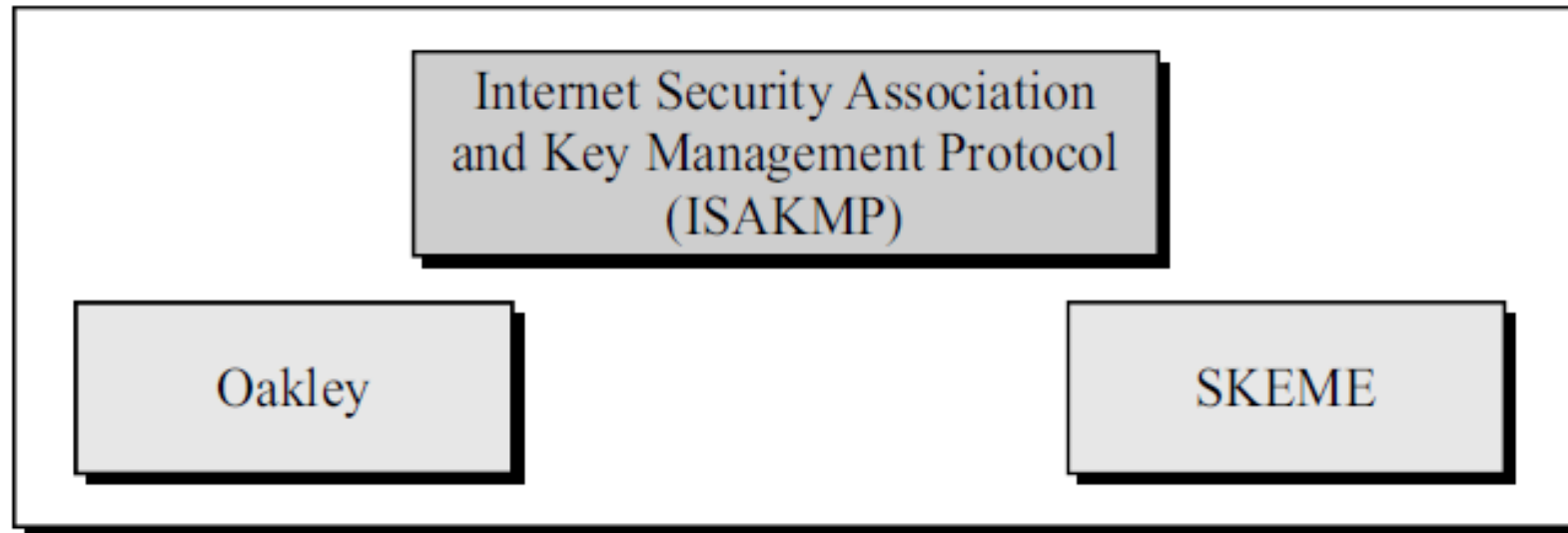
- **The Internet Key Exchange (IKE) is a protocol designed to create both inbound and outbound Security Associations.**
- **When a peer needs to send an IP packet, it consults the Security Policy Database (SPDB) to see if there is an SA for that type of traffic.**
- **If there is no SA, IKE is called to establish one.**

IKE creates SAs for IPsec.

- IKE is a complex protocol based on three other protocols: **Oakley, SKEME, and ISAKMP**

IKE components

Internet Key Exchange (IKE)



1. Oakley Protocol:

- Developed by Hilarie Orman.
- A key creation protocol based on the **Diffie–Hellman key exchange**.
- Provides improvements over basic Diffie–Hellman:
 - ✓ **Protection against replay attacks**
 - ✓ **Better authentication methods**
- Free-formatted protocol: Does not define fixed message formats
- Oakley concepts are used by IKE, though Oakley itself is not used directly.

2. SKEME Protocol:

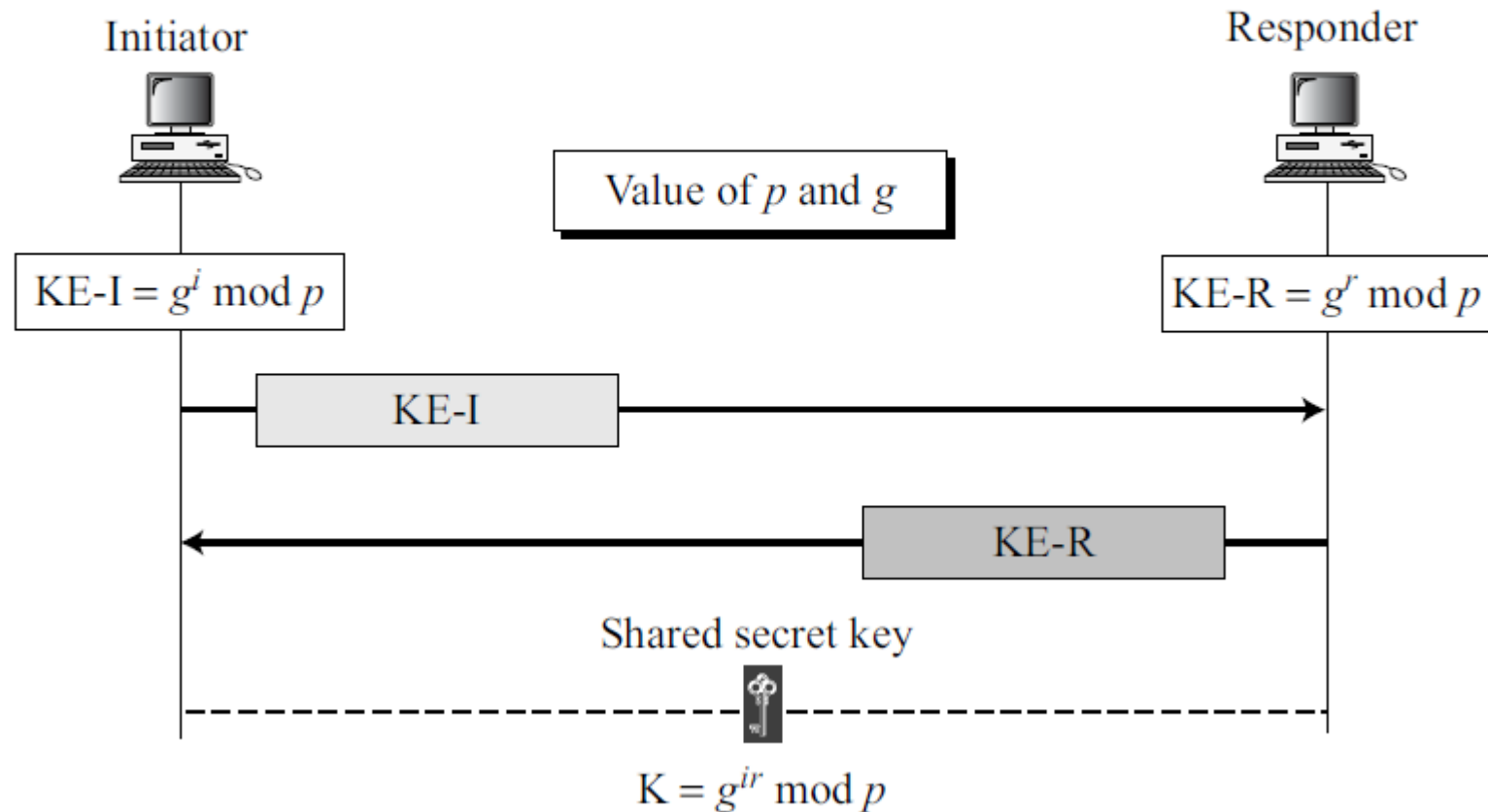
- Designed by Hugo Krawczyk.
- **A key exchange protocol.**
- **Uses public-key encryption for entity authentication.**
- One of the authentication methods in IKE is based on SKEME.

ISAKMP (Internet Security Association and Key Management Protocol)

- Designed by the **National Security Agency (NSA)**.
- Provides a **framework** for:
 - Packet formats
 - Message types
 - Exchange procedures
- Acts as a **carrier protocol** for IKE.
- Enables **standardized and structured exchanges** to create SAs.
- ISAKMP itself **does not define key exchange algorithms**.

Diffie-Hellman Key Exchange

18.16 *Diffie-Hellman key exchange*



- Diffie–Hellman is used to create a shared symmetric session key between two parties.
- The key is used only for the current session and is not stored for future use.
- **Parameters Used**
 - **p**: A large prime number
 - About **1024 bits (≈ 300 decimal digits)**
 - **g**: A generator of the group $\langle \mathbb{Z}_p^*, \times \rangle$
 - These values are **publicly known**.

Key Exchange Process

Alice chooses a large random number i .

- Computes: $KE-I = g^i \bmod p$
- Sends KE-I to Bob.

Bob chooses a large random number r .

- Computes: $KE-R = g^r \bmod p$
- Sends KE-R to Alice.

Key Generation

- Alice computes: $K = (KE-R)^i \bmod p = g^{ir} \bmod p$
- Bob computes: $K = (KE-I)^r \bmod p = g^{ir} \bmod p$

Both obtain the same symmetric session key K .

Weaknesses of Diffie–Hellman

1. Clogging attack

2. Replay attack

3. Man-In-The-Middle attack

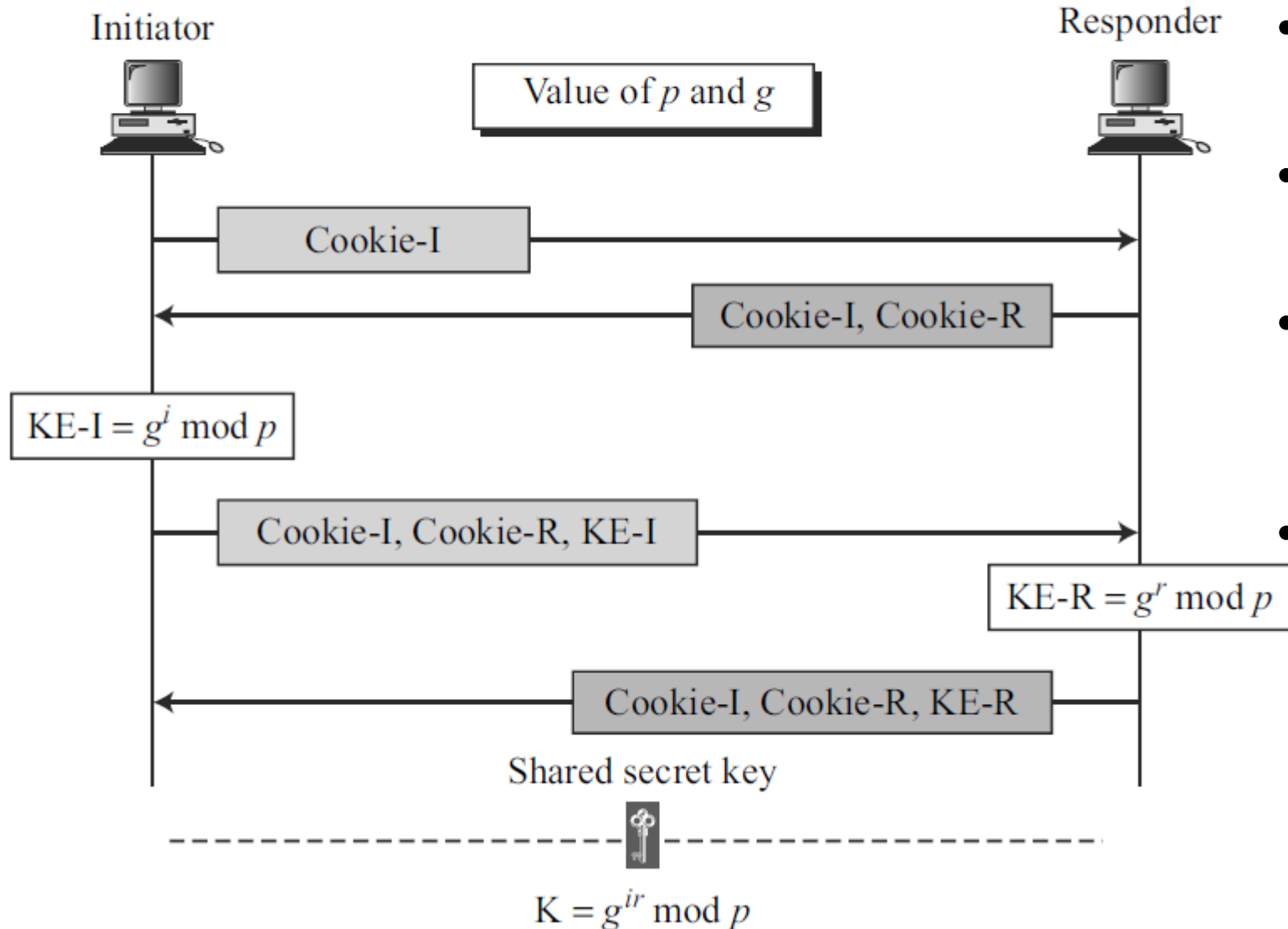
Clogging (Denial-of-Service) Attack in Diffie–Hellman

- A clogging attack is a type of Denial-of-Service (DoS) attack.
- **An attacker sends many fake Diffie–Hellman half-keys to a victim** (e.g., Bob).
- Each half-key looks like: $g^x \bmod q$
- Diffie–Hellman computations are computationally expensive.
 - ✓ Bob must: Generate responses $g^y \bmod q$
 - ✓ Compute full session keys $g^{xy} \bmod q$
 - ✓ Continuous fake requests consume CPU and memory resources.
- As a result, Bob fails to serve legitimate clients.
- **Effect of the Attack**
 - ✓ Bob becomes **overloaded**.
 - ✓ Legitimate users are **denied service**

Prevention of Clogging Attack Using Cookies:

- **Cookies are lightweight values** used to verify the peer before heavy computation.
- They are created using a **hash function**.
- A cookie is generated by hashing:
 - ✓ **Peer's IP address**
 - ✓ **Port number**
 - ✓ **Protocol**
 - ✓ **A secret random number** (known only to the generator)
 - ✓ **A timestamp**

8.17 Diffie-Hellman with cookies



- Initiator sends a request with its cookie.
- Responder replies with its own cookie.
- Both parties repeat the cookies unchanged in all further messages.
- Diffie-Hellman computations are postponed until cookies are verified.

Replay Attack in Diffie–Hellman

- In a replay attack, an attacker captures valid messages from an old session.
- The attacker re-sends (replays) these messages in a later session.
- Since Diffie–Hellman messages look valid, the receiver may accept them.

Prevention of Replay Attack

- **Nonces** are added to the protocol.
- A nonce is a **random number used only once**.
- It is **generated fresh for every session**.
- Nonces are included in the third and fourth messages.
- Each party sends a nonce and expects it to be returned correctly.
- If an attacker replays an old message:
 - ✓ The nonce will be outdated or incorrect.
 - ✓ The receiver rejects the message.
- This **ensures message freshness and prevents reuse of old messages**.

Man-in-the-Middle (MITM) Attack

- Diffie–Hellman does not authenticate identities.
- Anyone can send half-keys pretending to be another party.
- Thus, third person can: Read all messages, Modify messages and Forward messages without being detected.
- This is the most dangerous attack on Diffie–Hellman.

Preventing Man-in-the-Middle Attack

- Both parties must:
 - Verify **message integrity**
 - Authenticate **each other's identity**
- Each party must **prove possession of a secret**.

Authentication Methods Used in IKE

1. Pre-shared Secret Key

- Both parties already share a **secret key**.
- Messages are authenticated using this shared key.

2. Public-Key Encryption Pair

- Each party has a **public/private key pair**.
- A party proves identity by decrypting a message encrypted with its public key.

3. Digital Signature Key Pair

- Each party has a **signing key pair**.
- A party proves identity by:
 - Signing a message with its private key
 - The other party verifies it using the public key.

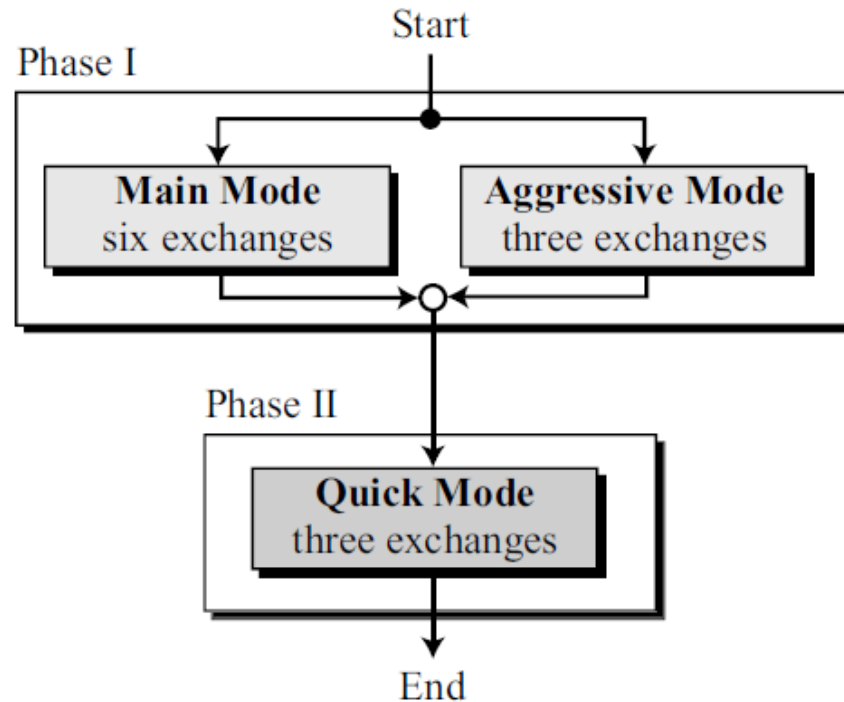
IKE Phases

- IKE operates on two phases:

Phase 1:

Phase 2:

IKE Phases



Phase 1

- Purpose: **Create SAs for Phase II**
- **Establishes a secure, authenticated channel between two peers**
- This secure channel is later used by **Phase 2** to create Security Associations (SAs) for IPSec.
- Phase 1 ensures that:
 - ✓ **The communicating peers are authenticated**
 - ✓ **A shared secret key is generated**
 - ✓ **A secure IKE SA is created**

Functions of IKE Phase 1

IKE Phase 1 performs the following functions:

- 1. Negotiates IKE Security Association (SA) parameters**
- 2. Authenticates the two peers**
- 3. Performs Diffie–Hellman key exchange**
- 4. Generates shared secret keys**
- 5. Protects against replay and denial-of-service attacks**

Phase 1 Modes

- IKE Phase 1 operates in **two modes**:

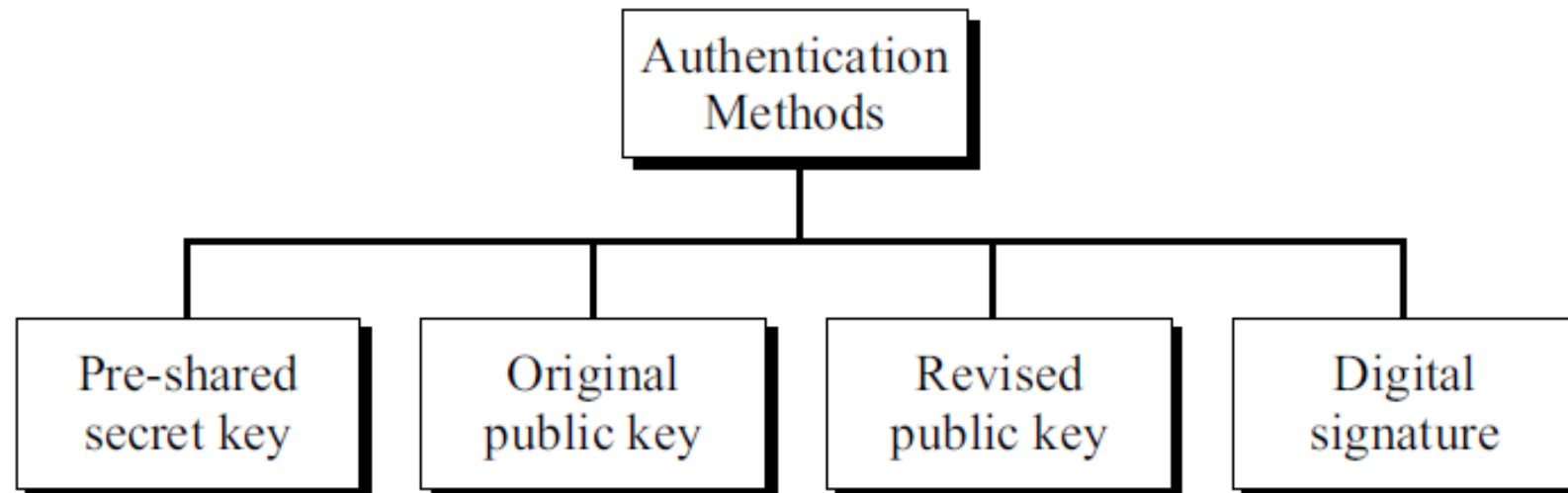
1. **Main Mode**

- ✓ Uses **six message exchanges**
- ✓ Provides **strong security**
- ✓ Hides the identities of the peers
- ✓ Commonly used in practice

2. **Aggressive Mode**

- ✓ Uses **three message exchanges**
- ✓ Faster but less secure
- ✓ Peer identities may be exposed

9 *Main-mode or aggressive-mode methods*



Phase 1 – Main Mode Operation

- Main mode consists of **six messages exchanged** between the initiator and responder.

Messages 1 and 2: SA Negotiation

- Initiator and responder exchange **cookies**
- Security Association (SA) parameters are negotiated
 - ✓ Initiator sends multiple proposals
 - ✓ Responder selects one proposal
- Cookies protect against **clogging (DoS) attacks**
- After this step: **Both parties agree on encryption, hashing, and authentication algorithms**

Messages 3 and 4: Key Exchange

- Exchange of:
 - **Diffie–Hellman half keys (g^i and g^r)**
 - **Nonces**
- Nonces provide protection against **replay attacks**
- Diffie–Hellman allows both parties to compute a **shared secret**
- After this step: **Both peers independently compute the same secret key**
- From the shared secret, the following keys are derived:
 1. **SKEYID – base secret key**
 2. **SKEYID_d – used to derive additional keys**
 3. **SKEYID_a – used for authentication**
 4. **SKEYID_e – used for encryption**
- These keys secure the remaining exchanges.

Messages 5 and 6: Authentication

- Initiator sends **HASH-I**
- Responder sends **HASH-R**
- Hash values prove the identity of each peer
- Messages are **encrypted using SKEYID_e**
- **If the received hash matches the calculated hash, authentication is successful.**

Authentication Methods in Phase 1

Phase 1 supports four authentication methods:

- 1. Preshared secret-key authentication**
- 2. Original public-key authentication**
- 3. Revised public-key authentication**
- 4. Digital signature authentication**

These methods differ in how SKEYID and authentication hashes are calculated.

Figure 18.20 *Main mode, preshared secret-key method*

KE-I (KE-R): Initiator's (responder's) half-key

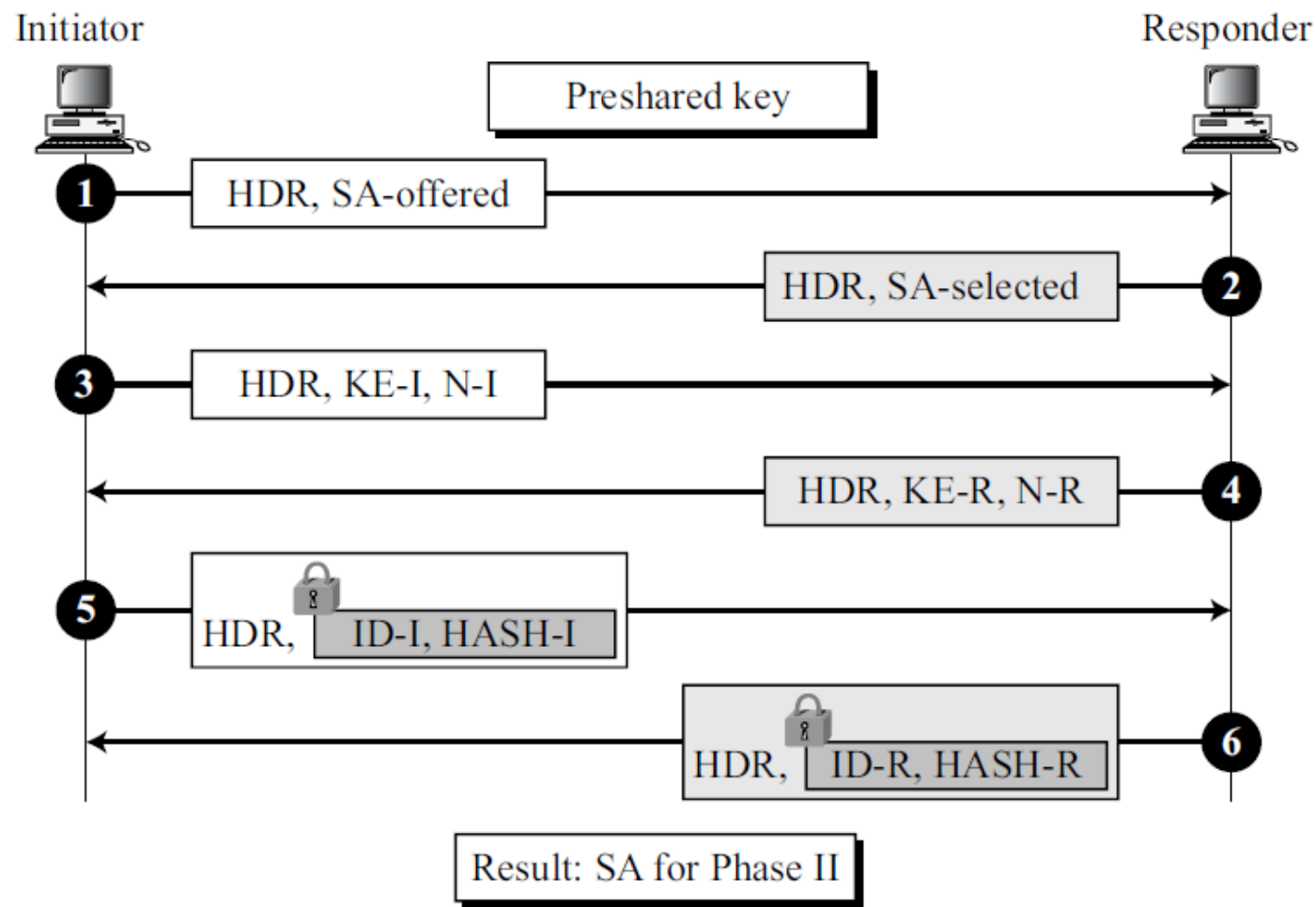
N-I (N-R): Initiator's (responder's) nonce

ID-I (ID-R): Initiator's (responder's) ID

HASH-I (HASH-R): Initiator's (responder's) hash

HDR: General header including cookies

 Encrypted with SKEYID_e



Message 1 (Initiator → Responder)

- HDR, SA-offered
- Initiator proposes security options:
 - ✓ Encryption algorithm
 - ✓ Hash algorithm
 - ✓ Authentication method
 - ✓ Diffie-Hellman group
- Cookie is included to prevent DoS (clogging attack)

Message 2 (Responder → Initiator)

- HDR, SA-selected
- Responder selects one proposal from the offered list
- Confirms SA parameters

Message 3 (Initiator → Responder)

- HDR, KE-I, N-I
- Initiator sends:
 - Its Diffie-Hellman half-key (KE-I)
 - Its nonce (N-I)

Message 4 (Responder → Initiator)

- HDR, KE-R, N-R
- Responder sends:
 - Its Diffie-Hellman half-key (KE-R)
 - Its nonce (N-R)
- Now both sides can compute: Shared Diffie-Hellman key
- SKEYID, SKEYID_a, SKEYID_e

Message 5 (Initiator → Responder)

- **HDR, ID-I, HASH-I (encrypted)**
- Initiator sends:
 - Its identity (**ID-I**)
 - Authentication hash (**HASH-I**)
- Message is encrypted using **SKEYID_e**
- Initiator knows the **preshared secret key**
- **Initiator is authentic**
- Authentication hash (**HASH-R**)
- Also encrypted with **SKEYID_e**
- Proves responder's identity
- **Final Result: Secure SA for Phase 2 is created**
- Both sides are:
 - ✓ Authenticated
 - ✓ Protected from replay and MITM attacks

Message 6 (Responder → Initiator)

- **HDR, ID-R, HASH-R (encrypted)**
- Responder sends:
 - Its identity (**ID-R**)
- Phase 2 (Quick Mode) can now create **IPSec SAs**

ire 18.21 Main mode, original public-key method

HDR: General header including cookies

KE-I (KE-R): Initiator's (responder's) half-key

N-I (N-R): Initiator's (responder's) nonce

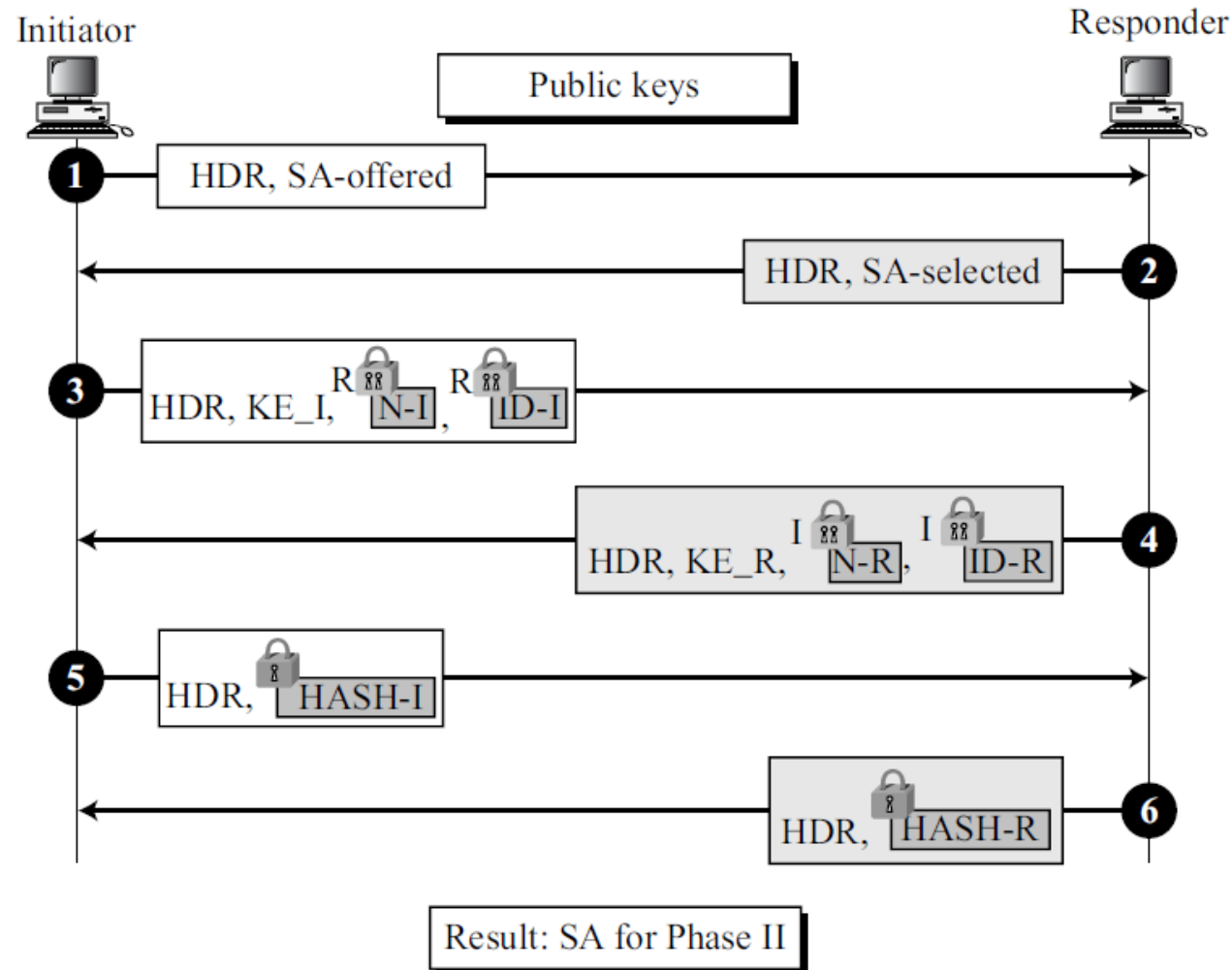
ID-I (ID-R): Initiator's (responder's) ID

HASH-I (HASH-R): Initiator's (responder's) hash

I  Encrypted with initiator's public key

R  Encrypted with responder's public key

 Encrypted with SKEYID_e



Message 1 (Initiator → Responder)

- **HDR, SA-offered**
- Initiator proposes:
 - ✓ Encryption, hashing algorithms
 - ✓ DH group
 - ✓ Authentication method (public-key)
- Cookie included (DoS protection)

Message 3 (Initiator → Responder)

- **HDR, KE-I, {N-I, ID-I}^R**
- Initiator sends:
 - ✓ Diffie-Hellman half-key (**KE-I**) → not encrypted
 - ✓ Nonce (**N-I**) → encrypted with responder's public key

✓ Identity (**ID-I**) → encrypted with responder's public key

- Only responder can decrypt N-I and ID-I using its **private key**

Message 4 (Responder → Initiator)

- **HDR, KE-R, {N-R, ID-R}^I**
- Responder sends:
 - ✓ Diffie-Hellman half-key (**KE-R**) → not encrypted
 - ✓ Nonce (**N-R**) → encrypted with initiator's public key
 - ✓ Identity (**ID-R**) → encrypted with initiator's public key
- Only initiator can decrypt these

What happens after messages 3 & 4

- Both sides:
 - ✓ Compute Diffie-Hellman shared secret
 - ✓ Derive **SKEYID_a**, **SKEYID_e**
- Authentication is now possible

Message 5 (Initiator → Responder) HDR, HASH-I

- Initiator sends **HASH-I**
- Encrypted with **SKEYID_e**
- Proves:

1. Initiator decrypted responder's nonce

2. Initiator owns the correct private key

Message 6 (Responder → Initiator)

HDR, HASH-R

- Responder sends **HASH-R**
- Encrypted with **SKEYID_e**
- Proves responder's identity

Final Result

- ✓ Secure SA for Phase 2 is established
- ✓ Mutual authentication completed
- ✓ Keys are ready for IPSec SA creation

gure 18.22 Main mode, revised public-key method

- HDR: General header including cookies

KE-I (KE-R): Initiator's (responder's) half-key

Cert-I (Cert-R): Initiator's (responder's) certificate

N-I (N-R): Initiator's (responder's) nonce

ID-I (ID-R): Initiator's (responder's) ID

HASH-I (HASH-R): Initiator's (responder's) hash
- I

Encrypted with initiator's public key

R

Encrypted with responder's public key

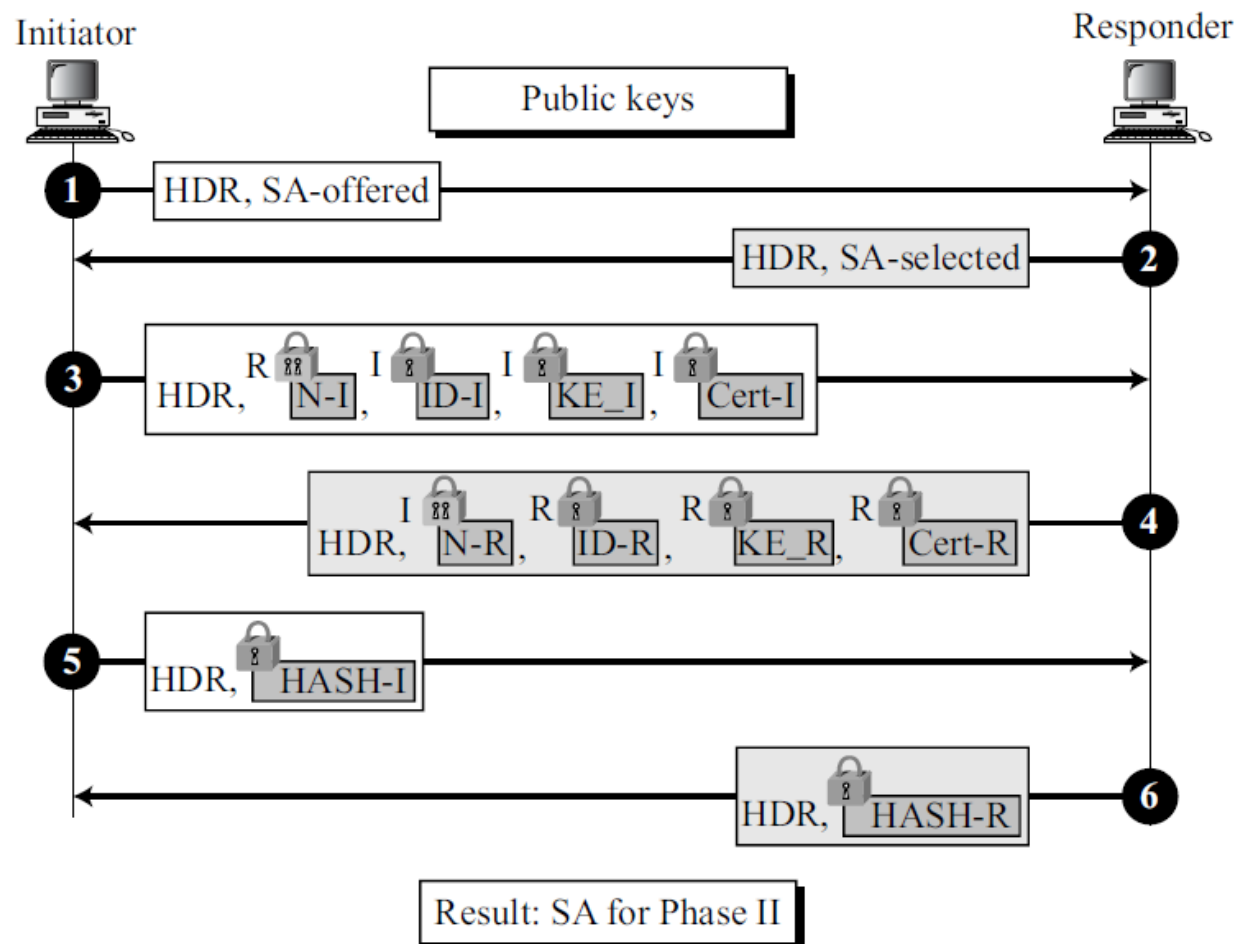
R

Encrypted with responder's secret key

I

Encrypted with initiator's secret key

Encrypted with SKEYID_e



Message 1 (Initiator → Responder)

- **HDR, SA-offered**
- Initiator proposes:
 - ✓ Algorithms
 - ✓ DH group
 - ✓ Authentication method
- Cookies included

Message 2 (Responder → Initiator)

- **HDR, SA-selected**
- Responder selects one proposal
- SA parameters fixed

Message 3 (Initiator → Responder)

- **HDR, {N-I}^R, {ID-I, KE-I, Cert-I}^I**
- Initiator sends:
 - **Nonce (N-I)** → encrypted with

responder's public key

- Uses N-I to derive **temporary secret key K-I**
- **ID-I, KE-I, Certificate** → encrypted with **K-I**
- Public key used **only once**

Message 4 (Responder → Initiator)

- **HDR, {N-R}^I, {ID-R, KE-R, Cert-R}^R**
- Responder sends:
 - **Nonce (N-R)** → encrypted with initiator's public key
 - Uses N-R to derive **temporary secret key K-R**
 - **ID-R, KE-R, Certificate** → encrypted with **K-R**

What happens after messages 3 & 4

- Both sides:
 - Exchange nonces
 - Create temporary secret keys
 - Compute Diffie-Hellman shared secret
 - Derive:
 - ✓ SKEYID
 - ✓ SKEYID_a
 - ✓ SKEYID

Message 5 (Initiator → Responder)

- HDR, HASH-I
- Encrypted using SKEYID_e

- Proves initiator's identity

Message 6 (Responder → Initiator)

- HDR, HASH-R
- Encrypted using SKEYID_e
- Proves responder's identity

Final result

- Secure SA for Phase II is created
- Mutual authentication completed
- Keys ready for IPSec

figure 18.23 *Main mode, digital signature method*

HDR: General header including cookies

Sig-I: Initiator's signature on messages 1–4

Sig-R: Initiator's signature on messages 1–5

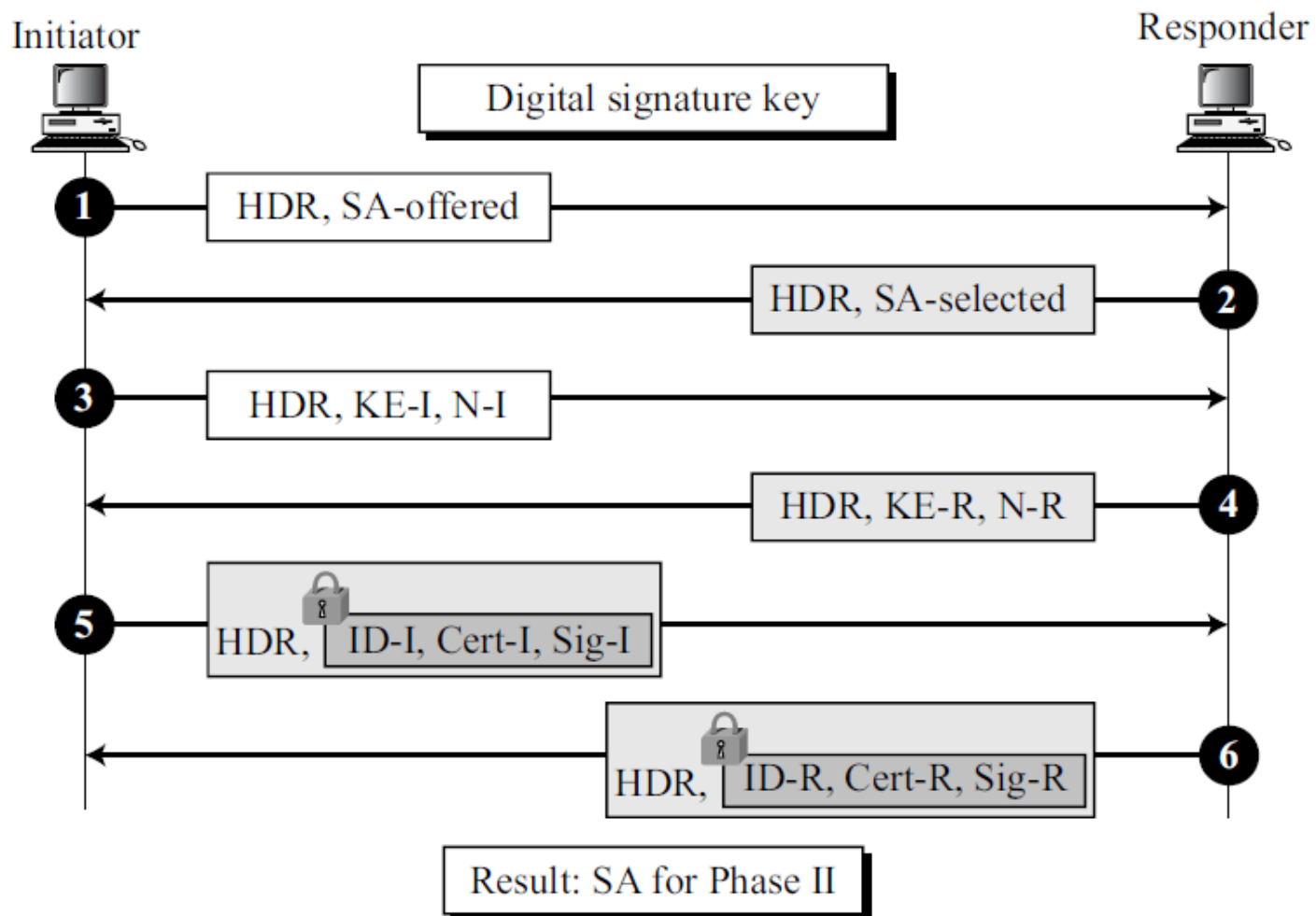
Cert-I (Cert-R): Initiator's (responder's) certificate

N-I (N-R): Initiator's (responder's) nonce

KE-I (KE-R): Initiator's (responder's) half-key

ID-I (ID-R): Initiator's (responder's) ID

 Encrypted with SKEYID_e



Message 1: Initiator → Responder

HDR, SA-offered

- Initiator proposes security parameters:
 - ✓ Encryption algorithm
 - ✓ Hash algorithm
 - ✓ Authentication method (digital signature)
 - ✓ Diffie–Hellman group

Message 2: Responder → Initiator

HDR, SA-selected

- Responder selects **one acceptable SA** from the proposal

Message 3: Initiator → Responder

HDR, KE-I, N-I

- Initiator sends:
 - ✓ **KE-I**: Diffie–Hellman public value (half-key)
 - ✓ **N-I**: Initiator's nonce
- Used later to generate shared secret keys.

Message 4: Responder → Initiator

HDR, KE-R, N-R

- Responder sends:
 - **KE-R**: Its Diffie–Hellman public value
 - **N-R**: Responder's nonce
- Now both sides can compute the same shared secret.
- At this point:
 - ✓ Shared secret is computed
 - ✓ **SKEYID**, **SKEYID_e** are derived
 - ✓ Encryption is now possible

Message 5: Initiator → Responder

(Encrypted): HDR, ID-I, Cert-I, Sig-I

- Initiator sends:
 - ✓ **ID-I**: Its identity
 - ✓ **Cert-I**: Its digital certificate

✓ **Sig-I**: Digital signature over messages 1–4

Proves:

- ✓ Initiator's identity
- ✓ Ownership of private key
- ✓ Integrity of earlier messages

Message 6: Responder → Initiator

(Encrypted): HDR, ID-R, Cert-R, Sig-R

- Responder sends:
 - ✓ Its identity
 - ✓ Its certificate
 - ✓ Its digital signature over messages 1–5
- Proves responder's authenticity.

IKE Phase II – Quick Mode

- After **Phase I** (Main Mode or Aggressive Mode) establishes an **IKE Security Association (IKE SA)**, Phase II can begin.
- Phase II uses **Quick Mode**, which is the **only mode defined for Phase II**.

Purpose of Phase II:

- ✓ To create **IPsec Security Associations (IPsec SAs)**
- ✓ To generate **key material used by IPsec** (ESP/AH)
- ✓ To operate under the protection of the **IKE SA created in Phase I**

Characteristics of Quick Mode

- ✓ Runs under the supervision of Phase I IKE SA
- ✓ Uses **3 messages only**
- ✓ **Either party** (initiator or responder of Phase I) can initiate Phase II
- ✓ All messages are:
 - **Encrypted** using SKEYID_e
 - **Authenticated** using keyed-HMAC hashes

Message Exchange in Quick Mode

Message 1: Initiator → Responder

Contents:

- ✓ HASH1 (keyed-HMAC)
- ✓ IPsec SA proposal
- ✓ Initiator nonce N-I
- ✓ Optional Diffie–Hellman half-key KE-I
- ✓ Optional IDs of both parties

Purpose:

- Proposes IPsec parameters
- Initiates key generation
- Requests optional **Perfect Forward Security (PFS)**

Message 2: Responder → Initiator

Contents:

- ✓ HASH2 (keyed-HMAC)

✓ Responder nonce N-R

✓ Optional Diffie–Hellman half-key KE-R

Purpose:

- Confirms SA parameters
- Contributes to key generation
- Completes Diffie–Hellman exchange (if PFS is used)

Message 3: Initiator → Responder

Contents:

- ✓ HASH3 (keyed-HMAC)

Purpose:

- Confirms the entire exchange
- Finalizes creation of IPsec SAs

Authentication Using HASH Values

Quick Mode uses **three keyed-HMACs**:

1. **HASH1**
2. **HASH2**
3. **HASH3**

- Each HASH includes the **Message ID (MsgID)** from ISAKMP header
- MsgID allows **multiplexing**
- Prevents confusion when **multiple Phase II negotiations occur simultaneously**
- Ensures **message integrity and**

authentication

Encryption in Phase II

- All Quick Mode messages are encrypted using **SKEYID_e**
- Ensures:
 - ✓ Confidentiality
 - ✓ Protection of SA parameters and keys

Perfect Forward Security (PFS)

- Keys for Quick Mode are normally derived from SKEYID_d
- Multiple Phase II sessions can depend on one Phase I
- If SKEYID_d is compromised, all derived keys are at risk

How PFS Works

- An **additional Diffie–Hellman exchange** is performed in Quick Mode
- A **new shared secret g^{ir} is generated**
- g^{ir} is used in key material calculation for IPsec
- Diffie–Hellman values are **deleted immediately after use**
- This ensures compromise of one key does **not affect past or future keys**